

Justin Heng Zhang, Vlad Shkuratov, Aodon Bowie

Robert Vincent

Programming techniques and applications

May 6th 2025

Final Project

For our final project, we have decided to code an AI to help us place sports bets for tennis matches. To do so, we started by collecting data from all the tennis matches registered under the ATP (association of tennis professionals) dating from the years 2000 all the way to 2025. We then merged all these excel files into a .csv formatted table that stored all sorts of information that would help us predict the winner such as: who played against who, type of surface (clay, grass, etc...), tournament name, tournament date, winner or loser of the matchup , etc....We then cleaned up the .csv file by adding other calculated features that would enhance the prediction such as: win or loss streaks, previous head to head matchups between the same players, games played in the last 30 days and most importantly the elo system. We implemented an elo system that changes after each match. The player elo is calculated using this formula: $E_A = \frac{1}{1+10^{(R_B-R_A)/400}}$ where E is the probability of the player winning and R is the elo of the player. We then updated the player elo after the match with formula: $R'_A = R_A + K(S_A - E_A)$, Where K is a constant that determines how much elo fluctuates, and where S is the score of the game(1 = win , 0 = loss). We then trained our machine with the cleaned .csv file.

To train the machine, we predicted the tennis matches outcomes using the XGBoost Classifier, a powerful gradient-boosted decision tree algorithm known for its accuracy and performance. We used pandas for data manipulation, scikit-learn for model evaluation and data

splitting, and XGBoost for training the model. Evaluation included accuracy scores, classification reports, probability predictions, expected value (EV) analysis, and simulated ROI. The final model was saved using joblib for future use. This was our `baseline_model.py`. We then implemented a couple more features to enhance the accuracy in our `custom_model.py`. This script is heavier and requires a gpu to be ran. To enhance the baseline DecisionTree model, we replaced it with an XGBoost classifier using GPU acceleration, added extensive feature engineering (e.g. Elo, H2H, fatigue, momentum), performed automatic feature selection with RFECV, and optimized hyperparameters via grid search with early stopping. We also calibrated probabilities, incorporated Kelly staking for betting simulation, and visualized feature importances. The custom model allowed us to increase our confidence rate from 66% to 69%